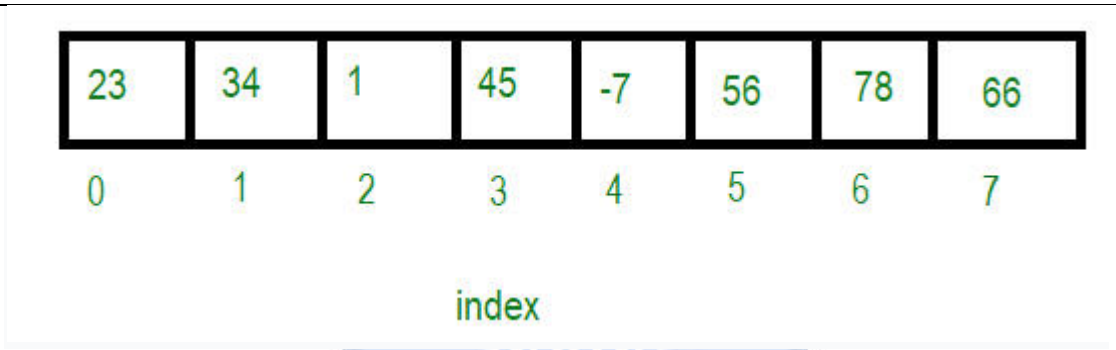
Subject Name: Programming for Problem Solving
Subject Code: BE01R00121

	CHAPTER NO- 4: ARRAY & STRING **(Solution for MSE-1 Syllabus Topics)**	
	TOPIC:1 Array: Concepts of Array, One and Two Dimensional Arrays, Declaration and Initialization of Arrays	
	SHORT QUESTIONS	
1	What is array? **[NLJIET]**	**1**
	An array is a collection of items of same data type stored at contiguous memory locations.	
2	int x[4] = {22,33,44}. What will be value of x[4]? **[NLJIET]**	**1**
	When element is not assigned as per the size of the array , it returns garbage value. The Garbage value is a random value at an address in the memory of a computer. Whenever a variable is defined without giving any value to it, it contains the leftover values from the previous program. The value of the memory of the computer can be anything unless a definite value is given to it.	
3	Give representation of 1D and 2D array. **[NLJIET]**	**2**
	One Dimensional Array in C We can visualize a one-dimensional array in C as a single row to store the elements. All the elements are stored at contiguous memory locations. The first element in the array has an index of 0, the second element has an index of 1, and so on.	



1-D Array Declaration

While declaring a one-dimensional array in C, the data type can be of any type, and also, we can give any name to the array, just like naming a random variable.

Syntax:

<data_type> <arr_name>[arr_size];

Two-Dimensional Array in C

A two-dimensional array or 2D array in C is the simplest form of the multidimensional array. We can visualize a two-dimensional array as an array of one-dimensional arrays arranged one over another forming a table with 'x' rows and 'y' columns where the row number ranges from 0 to (x-1) and the column number ranges from 0 to (y-1).

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

Graphical Representation of Two-Dimensional Array of Size 3 x 3

- **The syntax to declare the 2D array is given below.**

data_type array_name[rows][columns];

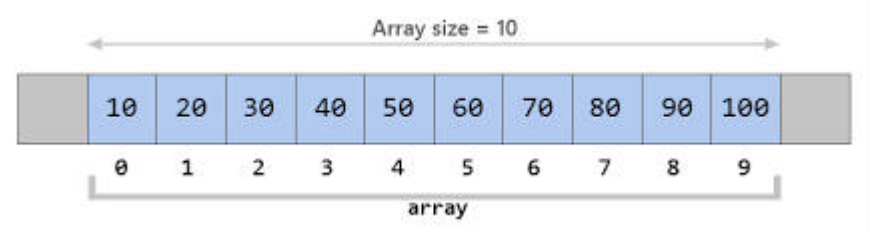
	DESCRIPTIVE QUESTIONS	
1	What is array? Explain initialization of an array value by example.**(Jan-2019-OLD[[NLJIET]** Discuss initialization of one-dimensional arrays with example.**(Jan-2017-OLD)[NLJIET]**	**3** **4**

What is an Array?

An array is a collection of one or more values of the same data type stored in contiguous memory locations. The data type can be user-defined or even any other primitive data-type. **Elements of an array can be accessed with the same array name by specifying the index number as the location in memory.**

One Dimensional Array in C

We can visualize a one-dimensional array in C as a single row to store the elements. All the elements are stored at contiguous memory locations.



Array Declaration

While declaring a one-dimensional array in C, the data type can be of any type, and also, we can give any name to the array, just like naming a random variable.

Syntax:

<data_type> <arr_name>[arr_size];

int arr[5]; *//arr is the array name of type integer, and 5 is the size of the array*

Array Initialization

In static uninitialized arrays, all the elements initially contain garbage values, but we can explicitly initialize them at their declaration.

Syntax:

<data_type> <arr_name> [arr_size]={value1, value2, value3,…};

Here, parameterized values are constant values separated by a comma.

We can skip the writing size of the array within square brackets if we initialize array elements explicitly within the list at the time of declaration. In that case, it will pick elements list size as array size.

Example:

int nums[5] = {0, 1, 2, 3, 4}; *//array nums is initialized with 5 elements 0,1,2,3,4*
int nums[] = {0, 1, 2, 3, 4}; *//array nums is initialized with size 5 as per the assigned element size in case of not mentioning size of array.*

Array Accessing

In one-dimensional arrays in C, elements are accessed by specifying the array name and the index value within the square brackets. **Array indexing starts from 0 and ends with size-1.** If we try to access array elements out of the range, the compiler will not show any error message; rather, it will return some garbage value.

Syntax:

<arr_name>[index];

Example:

int nums[5] = {0, 1, 2, 3, 4};
printf("%d", nums[0]); *//Array element at index 0 is printed*

printf("%d", nums[-1]); *//Garbage value will be printed*

C Program to illustrate declaration, initialization, and accessing of elements of a one-dimensional array in C:

```
#include < stdio.h >

int main() {
  //declaring and initializing one-dimensional array in C
  int arr[3] = {10, 20, 30};

   // After declaration, we can also initialize the array as:
   // arr[0] = 10; arr[1] = 20; arr[2] = 30;

   for (int i = 0; i < 3; i++) {
      // accessing elements of array
      printf(" Value of arr[%d]: %d\n", i, arr[i]);
   }
}
```

Output:

```
Value of arr[0]: 10
Value of arr[1]: 20
Value of arr[2]: 30
```

In this C programming code, we have initialized an array at the time of declaration with size 3 and array name as arr. At the end of the code, we are trying to print the array values by accessing its elements.

Rules for Declaring One Dimensional Array in C

Before using and accessing, we must declare the array variable.
In an array, indexing starts from 0 and ends at size-1. For example, if we have arr[10] of size 10, then the indexing of elements ranges from 0 to 9.
We must include data type and variable name while declaring one-dimensional arrays in C.

We can initialize them explicitly when the declaration specifies array size within square brackets is not necessary.
Each element of the array is stored at a contiguous memory location with a unique index number for accessing.

Types of Initialization of One-Dimensional Array in C

After declaration, we can initialize array elements or simply initialize them explicitly at the time of declaration. One-Dimensional arrays in C are initialized either at Compile Time or Run Time.

Compile-Time Initialization

Compile-Time initialization is also known as **static-initialization**. In this, array elements are initialized when we declare the array implicitly.

Syntax:

<data_type> <array_name> [array_size]={list of elements};

Example:

int nums[5] = {0, 1, 2, 3, 4};

C Program to illustrate Compile-Time Initialization:

```
#include <stdio.h>
int main(){
   int nums[3]={0,1,2};
   printf(" Compile-Time Initialization Example:\n");
   printf(" %d ",nums[0]);
   printf("%d ",nums[1]);
   printf("%d ",nums[2]);
}
```

Output:

0 1 2

In this C program code, we have initialized an array nums of size 3 and elements as 0,1 and 2 in the list. This is **compile-time initialization**, and then at the end, we have printed all its values by accessing index-wise.

Run-Time Initialization

Runtime initialization is also known as **dynamic-initialization**. Array elements are initialized at the runtime after successfully compiling the program.

Example:

scanf("%d", &nums[0]); *//initializing 0th index element at runtime dynamically*

C Program to illustrate Run-Time Initialization:

```
#include < stdio.h >

int main() {

   int nums[5];
   printf("\n Run-Time Initialization Example:\n");
   printf("\n Enter array elements: ");

   for (int i = 0; i < 5; i++) {
      scanf("%d", & nums[i]);
   }

   printf(" Accessing array elements after dynamic Initialization: ");

   for (int i = 0; i < 5; i++) {
      printf("%d ", nums[i]);
   }

   return 0;
```

}

Input

Run-Time Initialisation Example:

Enter array elements: 10 20 30 40 50

Output:

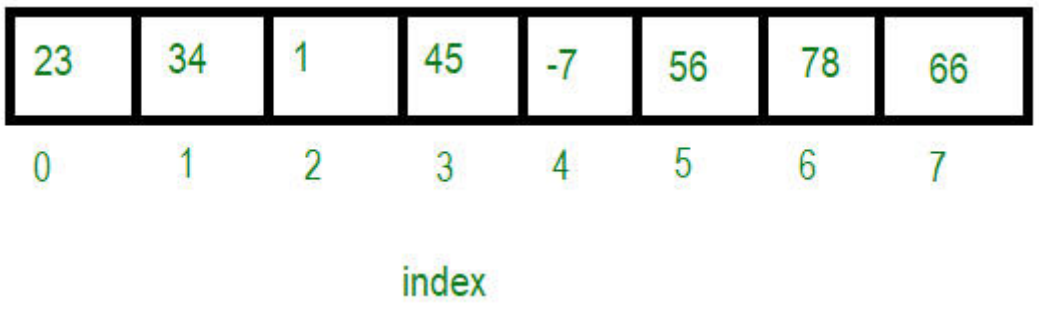
Accessing array elements after dynamic Initialization: 10 20 30 40 50

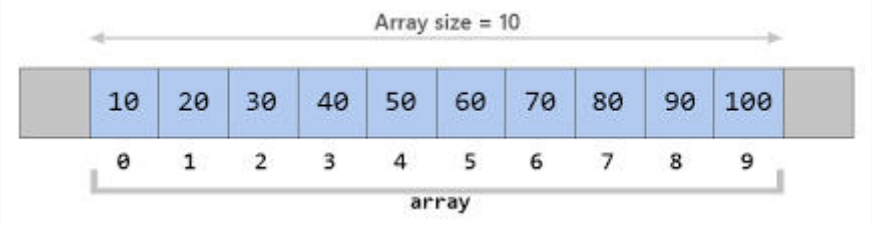
To demonstrate **runtime initialization**, we have just declared an array nums of size 5 in this C programming code. After that, within a loop, we ask the user to enter the array values to initialize them after compiling the code. In the end, we have printed its values by accessing them index-wise.

2	What is array? Give example of array. **(Jun-2017-OLD) [NLJIET]** What is array? Give example and advantages of array. **(May-2018-OLD) [NLJIET]** Describe array with example. **(Jan-2019-NEW) [NLJIET]**	**3** **3** **3**

- **An array is a data structure that stores a collection of elements, all of the same data type, in a contiguous block of memory. Each element in the array is identified by an index, which is a numerical value that represents the position of the element in the array.**
- The first element in the array has an index of 0, the second element has an index of 1, and so on.

Arrays can be used to store a variety of data types, including integers, floats, characters, and even user-defined types. They are commonly used to store collections of similar data, such as a list of numbers, a list of strings, or a list of objects.



	Though, the array got its own set of advantages and disadvantages. Advantages of Arrays Below are some advantages of the array: • In an array, accessing an element is very easy by using the index number. • The search process can be applied to an array easily. • 2D Array is used to represent matrices. • For any reason a user wishes to store multiple values of similar type then the Array can be used and utilized efficiently. • Arrays have low overhead. • C provides a set of built-in functions for manipulating arrays, such as sorting and searching. • C supports arrays of multiple dimensions, which can be useful for representing complex data structures like matrices. • Arrays can be easily converted to pointers, which allows for passing arrays to functions as arguments or returning arrays from functions.	
3	Show 1D array declaration, initialization and iteration. **(NOV-2020-NEW) [NLJIET]**	**3**
	One Dimensional Array in C We can visualize a one-dimensional array in C as a single row to store the elements. All the elements are stored at contiguous memory locations.  	

Array Declaration

While declaring a one-dimensional array in C, the data type can be of any type, and also, we can give any name to the array, just like naming a random variable.

Syntax:

<data_type> <arr_name>[arr_size];

int arr[5]; *//arr is the array name of type integer, and 5 is the size of the array*

Array Initialization

In static uninitialized arrays, all the elements initially contain garbage values, but we can explicitly initialize them at their declaration.

Syntax:

<data_type> <arr_name> [arr_size]={value1, value2, value3,…};

Here, parameterized values are constant values separated by a comma.

We can skip the writing size of the array within square brackets if we initialize array elements explicitly within the list at the time of declaration. In that case, it will pick elements list size as array size.

Example:

int nums[5] = {0, 1, 2, 3, 4}; *//array nums is initialized with 5 elements 0,1,2,3,4*
int nums[] = {0, 1, 2, 3, 4}; *//array nums is initialized with size 5 as per the assigned element size in case of not mentioning size of array.*

Array Accessing

In one-dimensional arrays in C, elements are accessed by specifying the array name and the index value within the square brackets. **Array indexing**

starts from 0 and ends with size-1. If we try to access array elements out of the range, the compiler will not show any error message; rather, it will return some garbage value.

Syntax:

<arr_name>[index];

Example:

```
int nums[5] = {0, 1, 2, 3, 4};
printf("%d", nums[0]); //Array element at index 0 is printed
printf("%d", nums[-1]); //Garbage value will be printed
```

C Program to illustrate declaration, initialization, and accessing of elements of a one-dimensional array in C:

```
#include < stdio.h >

int main() {
  //declaring and initializing one-dimensional array in C
  int arr[3] = {10, 20, 30};

   // After declaration, we can also initialize the array as:
   // arr[0] = 10; arr[1] = 20; arr[2] = 30;

   for (int i = 0; i < 3; i++) {
      // accessing elements of array
      printf(" Value of arr[%d]: %d\n", i, arr[i]);
   }
}
```

Output:

```
 Value of arr[0]: 10
 Value of arr[1]: 20
 Value of arr[2]: 30
```

In this C programming code, we have initialized an array at the time of declaration with size 3 and array name as arr. At the end of the code, we are trying to print the array values by accessing its elements.

Rules for Declaring One Dimensional Array in C

Before using and accessing, we must declare the array variable.
In an array, indexing starts from 0 and ends at size-1. For example, if we have arr[10] of size 10, then the indexing of elements ranges from 0 to 9.
We must include data type and variable name while declaring one-dimensional arrays in C.
We can initialize them explicitly when the declaration specifies array size within square brackets is not necessary.
Each element of the array is stored at a contiguous memory location with a unique index number for accessing.

Types of Initialization of One-Dimensional Array in C

After declaration, we can initialize array elements or simply initialize them explicitly at the time of declaration. One-Dimensional arrays in C are initialized either at Compile Time or Run Time.

Compile-Time Initialization

Compile-Time initialization is also known as **static-initialization**. In this, array elements are initialized when we declare the array implicitly.

Syntax:

<data_type> <array_name> [array_size]={list of elements};

Example:

int nums[5] = {0, 1, 2, 3, 4};

C Program to illustrate Compile-Time Initialization:

#include <stdio.h>

```
int main(){
   int nums[3]={0,1,2};
   printf(" Compile-Time Initialization Example:\n");
   printf(" %d ",nums[0]);
   printf("%d ",nums[1]);
   printf("%d ",nums[2]);
}
```

Output:

0 1 2

In this C program code, we have initialized an array nums of size 3 and elements as 0,1 and 2 in the list. This is **compile-time initialization**, and then at the end, we have printed all its values by accessing index-wise.

Run-Time Initialization

Runtime initialization is also known as **dynamic-initialization**. Array elements are initialized at the runtime after successfully compiling the program.

Example:

scanf("%d", &nums[0]); *//initializing 0th index element at runtime dynamically*

C Program to illustrate Run-Time Initialization:

```
#include < stdio.h >

int main() {

   int nums[5];
   printf("\n Run-Time Initialization Example:\n");
   printf("\n Enter array elements: ");

   for (int i = 0; i < 5; i++) {
```

```
        scanf("%d", & nums[i]);
    }

    printf(" Accessing array elements after dynamic Initialization: ");

    for (int i = 0; i < 5; i++) {
        printf("%d ", nums[i]);
    }

    return 0;
}
```

Input

Run-Time Initialisation Example:

Enter array elements: 10 20 30 40 50

Output:

Accessing array elements after dynamic Initialization: 10 20 30 40 50

To demonstrate **runtime initialization**, we have just declared an array nums of size 5 in this C programming code. After that, within a loop, we ask the user to enter the array values to initialize them after compiling the code. In the end, we have printed its values by accessing them index-wise.

4	Show 2D array declaration, initialization and iteration. **(NOV-2020-NEW) [NLJIET]** Demonstrate declaration and initialization of two dimensional array with suitable example. **(Mar-2022-NEW) [NLJIET]**	**3** **3**

- **Two Dimensional Array in C**

The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database lookalike data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

Declaration of two dimensional Array in C

- **The syntax to declare the 2D array is given below**.

data_type array_name[rows][columns];
Consider the following example.
int twodimen[4][3];
Here, 4 is the number of rows, and 3 is the number of columns.

- A 2D array is also known as a matrix (a table of rows and columns).

To create a 2D array of integers, take a look at the following example:

int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };

The first dimension represents the number of rows **[2]**, while the second dimension represents the number of columns **[3]**. The values are placed in row-order, and can be visualized like this:

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]

Initialization of 2D Array in C

In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously. However, this will not work with 2D arrays. We will have to define at least the second dimension of the array. The two-dimensional array can be declared and defined in the following way.

int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};

Two-dimensional array example in C

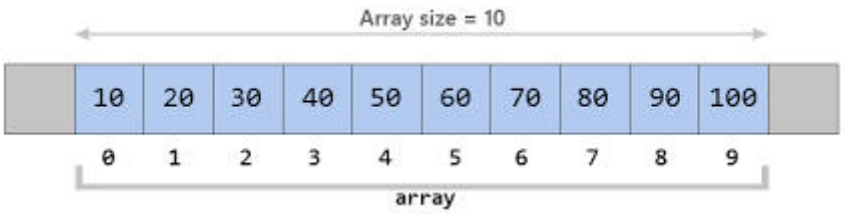
```
#include<stdio.h>
int main(){
int i=0,j=0;
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
//traversing 2D array
for(i=0;i<4;i++){
for(j=0;j<3;j++){
   printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);
 }//end of j
}//end of i
return 0;
}
```

Output

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6
```

5	What is an array? Explain one dimensional and two dimensional array declarations and initialization with suitable example. **(Jun-2019-NEW) [NLJIET]**	**7**
	An array is a data structure that stores a collection of elements, all of the same data type, in a contiguous block of memory. Each element in the array is identified by an index, which is a numerical value that represents the position of the element in the array. **<u>One Dimensional Array in C</u>**	

We can visualize a one-dimensional array in C as a single row to store the elements. All the elements are stored at contiguous memory locations.



Array Declaration

While declaring a one-dimensional array in C, the data type can be of any type, and also, we can give any name to the array, just like naming a random variable.

Syntax:

<data_type> <arr_name>[arr_size];

int arr[5]; *//arr is the array name of type integer, and 5 is the size of the array*

Array Initialization

In static uninitialized arrays, all the elements initially contain garbage values, but we can explicitly initialize them at their declaration.

Syntax:

<data_type> <arr_name> [arr_size]={value1, value2, value3,…};

Here, parameterized values are constant values separated by a comma.

We can skip the writing size of the array within square brackets if we initialize array elements explicitly within the list at the time of declaration. In that case, it will pick elements list size as array size.

Example:

int nums[5] = {0, 1, 2, 3, 4}; *//array nums is initialized with 5 elements 0,1,2,3,4*
int nums[] = {0, 1, 2, 3, 4}; *//array nums is initialized with size 5 as per the assigned element size in case of not mentioning size of array.*

Array Accessing

In one-dimensional arrays in C, elements are accessed by specifying the array name and the index value within the square brackets. **Array indexing starts from 0 and ends with size-1.** If we try to access array elements out of the range, the compiler will not show any error message; rather, it will return some garbage value.

Syntax:

<arr_name>[index];

Example:

```
int nums[5] = {0, 1, 2, 3, 4};
printf("%d", nums[0]); //Array element at index 0 is printed
printf("%d", nums[-1]); //Garbage value will be printed
```

C Program to illustrate declaration, initialization, and accessing of elements of a one-dimensional array in C:

```
#include < stdio.h >

int main() {
  //declaring and initializing one-dimensional array in C
  int arr[3] = {10, 20, 30};

   // After declaration, we can also initialize the array as:
   // arr[0] = 10; arr[1] = 20; arr[2] = 30;
```

```
    for (int i = 0; i < 3; i++) {
        // accessing elements of array
        printf(" Value of arr[%d]: %d\n", i, arr[i]);
    }
}
```

Output:

```
Value of arr[0]: 10
Value of arr[1]: 20
Value of arr[2]: 30
```

In this C programming code, we have initialized an array at the time of declaration with size 3 and array name as arr. At the end of the code, we are trying to print the array values by accessing its elements.

```
int main(){
    int nums[3]={0,1,2};
    printf(" Compile-Time Initialization Example:\n");
    printf(" %d ",nums[0]);
    printf("%d ",nums[1]);
    printf("%d ",nums[2]);
}
```

Output:

0 1 2

Two Dimensional Array in C

The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database lookalike data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

Declaration of two dimensional Array in C

- **The syntax to declare the 2D array is given below**.

data_type array_name[rows][columns];
Consider the following example.
int twodimen[4][3];
Here, 4 is the number of rows, and 3 is the number of columns.

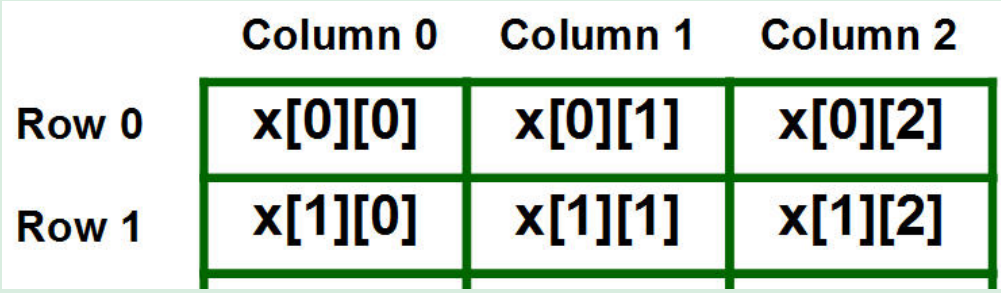
- A 2D array is also known as a matrix (a table of rows and columns).

To create a 2D array of integers, take a look at the following example:

int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };

The first dimension represents the number of rows **[2]**, while the second dimension represents the number of columns **[3]**. The values are placed in row-order, and can be visualized like this:

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8



Initialization of 2D Array in C
In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously. However, this will not work with 2D arrays. We will have to define at least the second dimension of the array. The two-dimensional array can be declared and defined in the following way.
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
Two-dimensional array example in C
#include<stdio.h>

	int main(){
	int i=0,j=0;
	int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
	//traversing 2D array
	for(i=0;i<4;i++){
	for(j=0;j<3;j++){
	printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);
	}//end of j
	}//end of i
	return 0;
	}
	Output
	arr[0][0] = 1
	arr[0][1] = 2
	arr[0][2] = 3
	arr[1][0] = 2
	arr[1][1] = 3
	arr[1][2] = 4
	arr[2][0] = 3
	arr[2][1] = 4
	arr[2][2] = 5
	arr[3][0] = 4
	arr[3][1] = 5
	arr[3][2] = 6	

6	Why it is necessary to give the size of an array in array declaration? **(Jun-2019-NEW) [NLJIET]**	**3**
	• We need to give the size of the array because the complier needs to allocate space in the memory which is not possible without knowing the size. • Compiler determines the size required for an array with the help of the number of elements of an array and the size of the data type present in the array.	
	PROGRAMS	

2	Write a program to find out the largest of an array. **(Jun-2017-OLD) (Mar-2022-NEW) [NLJIET]**	7

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[1000],i,max,n;
printf("Enter size of an array:");
scanf("%d",&n);
for(i=0;i<n;i++)
{
   printf("enter element: ");
   scanf("%d",&a[i]);
}
max=a[0];
for(i=0;i<n;i++)
{
   if(a[i]>max)
   {
   max=a[i];
   }

}
printf("\nLargest element is %d",max);
}
```

Output:

```
Enter size of an array:7
enter element: 1
enter element: 4
enter element: 2
enter element: 5
enter element: 3
enter element: 6
enter element: 8
```

	Largest element is 8	
3	Write a program to arrange the number of an array in ascending order. **(Jan-2019-OLD) [NLJIET]**	**7**

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[1000],i,temp,j,n;
printf("enter size of an array:");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("enter element: ");
scanf("%d",&a[i]);
}
for(i=0;i<n;i++)
{
   for(j=i+1;j<n;j++)
   {
      if(a[i]>a[j])
      {
         temp=a[i];
         a[i]=a[j];
         a[j]=temp;
      }

   }
}
printf("\n sorted array is: \n")
for(i=0;i<n;i++)
{
printf("\t %d",a[i]);
}

}
```

Output:

```
enter size of an array:6
enter element: 8
enter element: 5
enter element: 2
enter element: 4
enter element: 6
enter element: 2
sorted array is:
2 2 4 5 6 8
```

4 Write a program to store 10 elements in array given by user and to find maximum out of those 10 elements. **(Nov-2020-NEW) [NLJIET]** **7**

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10],i,max;
for(i=0;i<10;i++)
{
   printf("enter element: ");
   scanf("%d",&a[i]);
}
max=a[0];
for(i=0;i<10;i++)
{
   if(a[i]>max)
   {
   max=a[i];
   }

}
printf("\nLargest element is %d",max);
}
```

	Output: enter element: 3 enter element: 6 enter element: 9 enter element: 1 enter element: 4 enter element: 7 enter element: 2 enter element: 5 enter element: 8 enter element: 0 Largest element is 9	
5	Write a C program to read 10 numbers from user and store them in an array. Display Sum, Minimum and Average of the numbers. **(May-2018-OLD) (Jan-2020-NEW) [NLJIET]**	**7**

	#include<stdio.h>
	#include<conio.h>
	void main()
	{
	int a[1000],i,max,n,min;
	float sum=0,avg;
	printf("Enter size of an array:");
	scanf("%d",&n);
	for(i=0;i<n;i++)
	{
	printf("enter element: ");
	scanf("%d",&a[i]);
	sum=sum+a[i];
	}
	max=a[0];
	min=a[0];
	for(i=0;i<n;i++)
	{
	if(a[i]>max)	

```
    {
    max=a[i];
    }
    if(a[i]<min)
    {
    min=a[i];
    }

}
printf("\n sum is %f",sum);
avg=sum/n;
printf("\n Largest element is %d",max);
printf("\n smallest element is %d",min);
printf("\n average of all element is %f",avg);

}
```

Output:

```
Enter size of an array:10
enter element: 4
enter element: 1
enter element: 7
enter element: 8
enter element: 5
enter element: 2
enter element: 3
enter element: 6
enter element: 9
enter element: 7
 sum is 52.000000
 Largest element is 9
 smallest element is 1
 average of all element is 5.200000
```

5.1	Write a C program to get 20 numbers from user. Find sum, average, maximum and minimum number. **(Jun-2019-OLD) [NLJIET]**	7

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[1000],i,max,n,min;
float sum=0,avg;
printf("Enter size of an array:");
scanf("%d",&n);
for(i=0;i<n;i++)
{
   printf("enter element: ");
   scanf("%d",&a[i]);
   sum=sum+a[i];
}
max=a[0];
min=a[0];
for(i=0;i<n;i++)
{
   if(a[i]>max)
   {
   max=a[i];
   }
   if(a[i]<min)
   {
   min=a[i];
   }

}
printf("\n sum is %f",sum);
avg=sum/n;
printf("\n Largest element is %d",max);
printf("\n smallest element is %d",min);
printf("\n average of all element is %f",avg);
```

	} **Output:** Enter size of an array:20 enter element: 1 enter element: 2 enter element: 3 enter element: 6 enter element: 5 enter element: 4 enter element: 9 enter element: 8 enter element: 7 enter element: 11 enter element: 10 enter element: 12 enter element: 13 enter element: 14 enter element: 15 enter element: 16 enter element: 17 enter element: 18 enter element: 19 enter element: 20 sum is 210.000000 Largest element is 20 smallest element is 1 average of all element is 10.500000	
7	Write a C program to sort an array in ascending order.**(Mar-2023-NEW)[NLJIET]**	**7**
	#include<stdio.h> #include<conio.h>	

```
void main()
{
int a[1000],i,temp,j,n;
printf("enter size of an array:");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("enter element: ");
scanf("%d",&a[i]);
}
for(i=0;i<n;i++)
{
   for(j=i+1;j<n;j++)
   {
      if(a[i]>a[j])
      {
         temp=a[i];
         a[i]=a[j];
         a[j]=temp;
      }

   }
}
printf("\n sorted array is: \n")
for(i=0;i<n;i++)
{
printf("\t %d",a[i]);
}

}
```

Output:

```
enter size of an array:6
enter element: 8
enter element: 5
enter element: 2
```

	enter element: 4 enter element: 6 enter element: 2 sorted array is: 2 2 4 5 6 8	
8	Write a C program to make sum of array elements.**(Mar-2023-NEW)[NLJIET]**	**7**

	#include<stdio.h>
	#include<conio.h>
	void main()
	{
	int a[1000],i,sum=0,n;
	printf("enter size of array");
	scanf("%d",&n);
	for(i=0;i<n;i++)
	{
	printf("enter element: ");
	scanf("%d",&a[i]);
	sum=sum+a[i];
	}
	printf("\n sum of elements is %d",sum);
	}
	
	Output:
	enter size of array6
	enter element: 1
	enter element: 4
	enter element: 7
	enter element: 2
	enter element: 5
	enter element: 8
	
	sum of elements is 27	